

1 Roteiro Prático 05 — Assincronismo, Fetch API & Backend RESTful em Flask (SQLite)

Objetivos da Prática

Objetivos de Aprendizagem (Módulo 3 — Assincronismo & Integração de Serviços Web)

Ao final desta prática orientada em laboratório, o estudante será capaz de:

- Compreender a transição do armazenamento isolado no navegador (`localStorage`) para a **arquitetura Cliente-Servidor real**, onde os dados são centralizados e compartilhados em rede;
- Configurar e executar um microbackend RESTful em **Python Flask**, compreendendo o papel dos protocolos HTTP, rotas, verbos (GET, POST, PUT, DELETE) e códigos de status (200, 201, 400, 404, 500);
- Integrar um banco de dados relacional embutido (**SQLite 3**) via arquivo persistente (`vitrine.db`), eliminando a necessidade de instalar servidores externos de banco;
- Compreender o mecanismo de segurança **CORS** (*Cross-Origin Resource Sharing*) e por que ele bloqueia requisições entre portas distintas (ex: frontend na porta 5500 e API na porta 5000);
- Refatorar a camada de serviços (`vitrineService.js`) para operar com a moderna **Fetch API** do JavaScript nativo utilizando a sintaxe `async/await` e tratamento robusto de exceções com blocos `try/catch`;
- Implementar o padrão profissional de **Contingência Resiliente (Fallback)**: se a API do Flask estiver ativa (no laboratório), o sistema opera conectado ao banco relacional; se a página for executada no GitHub Pages ou offline, a aplicação chaveia automaticamente para o `localStorage` sem quebrar;
- Implementar estados visuais reativos no frontend com **Spinners de Carregamento** e **Badges dinâmicos de conectividade** com a API.

1.1 Introdução: Da Vitrine Isolada à Aplicação em Rede

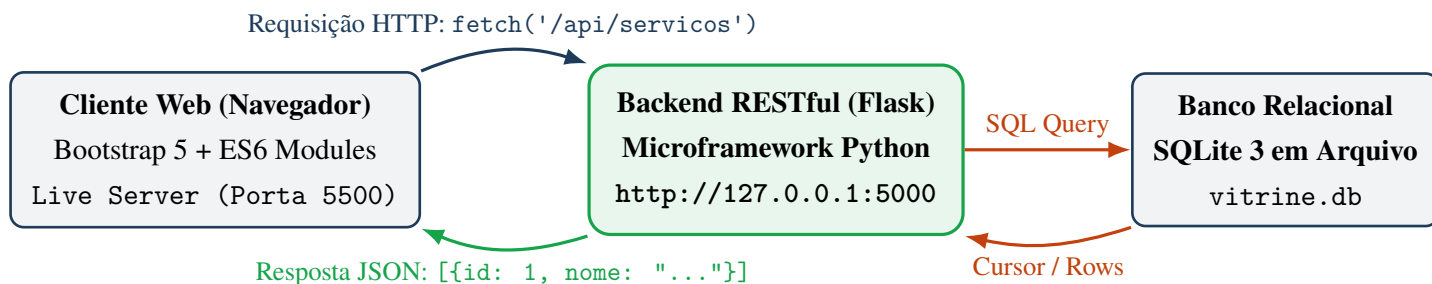
No **Roteiro Prático 04**, demos um grande salto organizando o frontend na arquitetura modular com ES6 Modules (`services/`, `views/`, `utils/`). No entanto, todo o armazenamento ocorria exclusivamente dentro do navegador de cada máquina através da Web Storage API (`localStorage`).

Na prática profissional, se um empreendedor em Garopaba cadastrar um novo serviço em seu celular, esse serviço precisa ficar imediatamente disponível para qualquer cliente que abrir a página em outro computador. Isso exige que os dados saiam do navegador e sejam gravados em um ****servidor**

central com banco de dados**.

Neste **Roteiro Prático 05**, evoluiremos a mesma base de código da **Vitrine Comunitária**, conectando o frontend modular a uma API RESTful leve construída em **Flask** com banco de dados relacional **SQLite**.

1.2 Visão Geral da Arquitetura Cliente-Servidor



1.3 Estrutura de Pastas do Projeto Integrado

Para manter o desacoplamento saudável entre a camada de backend e a camada de frontend, organizaremos o projeto na pasta `roteiro-5/exercicio-roteiro5/` com a seguinte estrutura:

Árvore de Diretórios do Projeto Integrado (Full-Stack Didático):

```
roteiro-5/exercicio-roteiro5/
|-- backend/
|   |-- requirements.txt  <-- Dependências Python (Flask, flask-cors)
|   |-- app.py           <-- API RESTful com rotas e conexão SQLite
|   |-- vitrine.db       <-- Banco SQLite (gerado automaticamente no 1º run)
|-- frontend/
|   |-- index.html       <-- Interface com status de conexão e spinner
|   |-- styles.css       <-- Estilos complementares
|   |-- js/
|       |-- main.js      <-- Orquestrador assíncrono de eventos
|       |-- services/
|           |-- vitrineService.js <-- Consumo Fetch API com fallback
|       |-- views/
|           |-- vitrineView.js  <-- Renderizador de cards, spinner e toasts
|       |-- utils/
|           |-- formatters.js  <-- Formatações (100% inalterado!)
```

Princípio do Desacoplamento Arquitetural

Observe atentamente que a camada utilitária (`formatters.js`) e grande parte da camada de visão (`vitrineView.js`) **permanecem exatamente as mesmas do Roteiro 04**. Toda a migração de tecnologia ocorre de forma isolada no `vitrineService.js`. Este é o maior benefício do padrão arquitetural que aprendemos!

1.4 Fundamentação Teórica: Conceitos-Chave

1.4.1 1. O Ciclo de Vida HTTP e Verbos REST

Uma API RESTful utiliza o protocolo HTTP padrão para realizar operações sobre recursos:

- **GET /api/servicos:** Solicita a leitura da lista de serviços cadastrados (*Read*). Não altera o estado do servidor. Retorna código de status 200 OK.
- **POST /api/servicos:** Envia um objeto JSON no corpo (*Body*) da requisição para cadastrar um novo serviço (*Create*). Retorna código 201 Created.
- **PUT /api/servicos/<id>:** Atualiza os dados de um registro existente pelo seu identificador (*Update*). Retorna código 200 OK.
- **DELETE /api/servicos/<id>:** Remove permanentemente um serviço do banco (*Delete*). Retorna código 200 OK.

1.4.2 2. A Política de Mesma Origem e o CORS

Por questões de segurança, os navegadores web impedem por padrão que um script executado em uma origem (ex: `http://127.0.0.1:5500` no Live Server) leia dados de outra origem (ex: `http://127.0.0.1:5000` no Flask). Essa política de bloqueio é chamada de **SOP** (*Same-Origin Policy*).

Para autorizar que o nosso frontend acesse a API, o servidor Flask precisa enviar nos cabeçalhos de resposta a diretiva **CORS** (*Cross-Origin Resource Sharing*):

```
Access-Control-Allow-Origin: *
```

No Python, essa autorização é resolvida de maneira elegante através da extensão **flask-cors**.

1.4.3 3. Assincronismo com Promises e async/await

No JavaScript, operações de rede (como buscar dados em outro computador) levam alguns milissegundos ou segundos para responder. Se o código executasse de forma síncrona (*bloqueante*), o navegador congelaria completamente enquanto esperasse a resposta.

Por isso, o JavaScript utiliza **Promises** (promessas de valores futuros) manipuladas pelas palavras-chave **async** e **await**:

```
// A palavra 'async' declara que a função lida com operações assíncronas
export async function obterDados() {
  try {
    // A palavra 'await' pausa a execução DESTA função até a resposta chegar,
    // permitindo que o navegador continue fluido e responsivo para o usuário.
    const resposta = await fetch('http://127.0.0.1:5000/api/servicos');

    // Converte o fluxo de texto HTTP em um objeto JavaScript
    const dados = await resposta.json();
  }
}
```

```
    return dados;
  } catch (erro) {
    console.error('Falha de rede:', erro);
  }
}
```

1.5 Implementação Passo a Passo Guiada

Seguiremos a sequência didática em 6 fases de implementação estruturadas. Todo o código fornecido é **100% completo**, sem reticências ou omissões.

Fase 1/6: Configuração do Backend em Flask & SQLite (backend/)

Responsabilidade Arquitetural: Criar o ambiente isolado Python (virtualenv), instalar o microframework Flask com suporte a CORS e implementar a API RESTful com conexão ao SQLite em arquivo (vitrine.db).

Passo 1.1: Criar o arquivo de dependências (backend/requirements.txt)

```
Flask>=3.0.0  
flask-cors>=4.0.0
```

Passo 1.2: No terminal do VS Code, criar o ambiente virtual e instalar dependências:

```
# Acesse a pasta do backend  
cd roteiro-5/exercicio-roteiro5/backend  
  
# Cria o ambiente virtual isolado  
python3 -m venv venv  
  
# Ativa o ambiente virtual (Linux/Mac)  
source venv/bin/activate  
# No Windows PowerShell utilize: .\venv\Scripts\Activate.ps1  
  
# Instala o Flask e o CORS  
pip install -r requirements.txt
```

Passo 1.3: Criar o arquivo completo do servidor (backend/app.py):

```
import os  
import sqlite3  
from datetime import datetime  
from flask import Flask, jsonify, request  
from flask_cors import CORS  
  
app = Flask(__name__)  
# Habilita CORS para permitir que o Live Server (porta 5500) acesse o Flask (5000)  
CORS(app)  
  
DB_PATH = os.path.join(os.path.dirname(__file__), 'vitrine.db')  
  
def obter_conexao():  
    """Conecta ao SQLite retornando linhas como dicionários."""  
    conn = sqlite3.connect(DB_PATH)  
    conn.row_factory = sqlite3.Row  
    return conn  
  
def inicializar_banco():  
    """Cria a tabela servicos e popula com os 3 registros modelo de Garopaba se vazia."""
```



```
conn = obter_conexao()
cursor = conn.cursor()
cursor.execute('''
    CREATE TABLE IF NOT EXISTS servicos (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nome TEXT NOT NULL,
        categoria TEXT NOT NULL,
        bairro TEXT NOT NULL,
        precoBase REAL NOT NULL,
        telefone TEXT NOT NULL,
        descricao TEXT,
        dataCadastro TEXT
    )
''')

cursor.execute('SELECT COUNT(*) AS total FROM servicos')
if cursor.fetchone()['total'] == 0:
    dados_semente = [
        (
            'Maré Alta Artesanatos & Cerâmicas', 'Artesanato', 'Centro Histórico',
            35.00, '48991234567',
            'Peças artesanais e utilitárias modeladas à mão com argila local e conchas de Garopaba.',
            datetime.now().isoformat()
        ),
        (
            'Garopaba Web & Design Studio', 'Tecnologia', 'Ferrugem',
            150.00, '48998765432',
            'Criação de websites profissionais responsivos, cardápios digitais e suporte para comércio local.',
            datetime.now().isoformat()
        ),
        (
            'Pescado Fresco do Zequinha', 'Alimentação', 'Canto das Canoas',
            42.00, '48984561234',
            'Peixes frescos e frutos do mar da pesca artesanal diária entregues com higiene e pontualidade.',
            datetime.now().isoformat()
        )
    ]
    cursor.executemany('''
        INSERT INTO servicos (nome, categoria, bairro, precoBase, telefone, descricao, dataCadastro)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    ''', dados_semente)
    conn.commit()
conn.close()

@app.route('/api/status', methods=['GET'])
def verificar_status():
    """Diagnóstico de conectividade com a API."""
    return jsonify({
        'status': 'online',
        'mensagem': 'API REST da Vitrine de Garopaba operacional.',
        'banco': 'SQLite 3'
    }), 200

@app.route('/api/servicos', methods=['GET'])
def listar_servicos():
    """READ: Retorna a lista de todos os serviços ordenados pelo ID decrescente."""
    conn = obter_conexao()
    cursor = conn.cursor()
```



```
cursor.execute('SELECT * FROM servicos ORDER BY id DESC')
linhas = cursor.fetchall()
conn.close()
return jsonify([dict(l) for l in linhas]), 200

@app.route('/api/servicos', methods=['POST'])
def cadastrar_servico():
    """CREATE: Grava um novo serviço enviado em formato JSON no SQLite."""
    dados = request.get_json()
    if not dados:
        return jsonify({'erro': 'Payload inválido.'}), 400

    conn = obter_conexao()
    cursor = conn.cursor()
    data_atual = datetime.now().isoformat()

    cursor.execute('''
        INSERT INTO servicos (nome, categoria, bairro, precoBase, telefone, descricao, dataCadastro)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    ''', (
        dados['nome'].strip(),
        dados['categoria'].strip(),
        dados['bairro'].strip(),
        float(dados['precoBase']),
        dados['telefone'].strip(),
        dados.get('descricao', '').strip(),
        data_atual
    ))
    conn.commit()
    novo_id = cursor.lastrowid
    conn.close()

    return jsonify({
        'id': novo_id,
        'nome': dados['nome'].strip(),
        'categoria': dados['categoria'].strip(),
        'bairro': dados['bairro'].strip(),
        'precoBase': float(dados['precoBase']),
        'telefone': dados['telefone'].strip(),
        'descricao': dados.get('descricao', '').strip(),
        'dataCadastro': data_atual
    }), 201

@app.route('/api/servicos/<int:id>', methods=['DELETE'])
def excluir_servico(id):
    """DELETE: Exclui um serviço pelo ID."""
    conn = obter_conexao()
    cursor = conn.cursor()
    cursor.execute('DELETE FROM servicos WHERE id = ?', (id,))
    conn.commit()
    conn.close()
    return jsonify({'mensagem': f'Serviço {id} removido.'}), 200

if __name__ == '__main__':
    inicializar_banco()
    print('Servidor Flask ativo na porta 5000: http://127.0.0.1:5000')
    app.run(host='127.0.0.1', port=5000, debug=True)
```

Fase 2/6: Executando e Testando a API RESTful

Responsabilidade Arquitetural: Iniciar o servidor Flask na porta 5000 e validar os endpoints de saúde (/api/status) e listagem (/api/servicos) diretamente no terminal e navegador antes da integração.

Passo 2.1: Iniciar o servidor no terminal:

```
python app.py
```

Passo 2.2: Testar no navegador ou via cURL em outra aba do terminal:

```
curl http://127.0.0.1:5000/api/status
```

Resposta esperada do servidor (JSON com status 200):

```
{"banco": "SQLite 3", "mensagem": "API REST da Vitrine de Garopaba operacional.", "status": "online"}
```

Checkpoint de Validação da Fase 2 Abra no navegador <http://127.0.0.1:5000/api/servicos>. Você deverá ver a lista JSON com os 3 registros semente cadastrados automaticamente no arquivo vitrine.db.

Fase 3/6: A Revolução Assíncrona no vitrineService.js

Responsabilidade Arquitetural: Refatorar os métodos de leitura e escrita para utilizarem requisições HTTP assíncronas com fetch() e async/await, mantendo uma camada de contingência transparente com localStorage.

```
// js/services/vitrineService.js
const API_BASE_URL = 'http://127.0.0.1:5000/api';
const STORAGE_KEY = 'garopaba_vitrine_servicos';

let apiConectada = false;

export const DADOS_INICIAIS = [
  {
    id: '1',
    nome: 'Maré Alta Artesanatos & Cerâmicas',
    categoria: 'Artesanato',
    bairro: 'Centro Histórico',
    precoBase: 35.00,
    telefone: '48991234567',
    descricao: 'Peças artesanais e utilitárias modeladas à mão com argila local e conchas de Garopaba.'
  },
  {
    id: '2',
```



```
    nome: 'Garopaba Web & Design Studio',
    categoria: 'Tecnologia',
    bairro: 'Ferrugem',
    precoBase: 150.00,
    telefone: '48998765432',
    descricao: 'Criação de websites profissionais responsivos, cardápios digitais e suporte para comércio local.'
  },
  {
    id: '3',
    nome: 'Pescado Fresco do Zequinha',
    categoria: 'Alimentação',
    bairro: 'Canto das Canoas',
    precoBase: 42.00,
    telefone: '48984561234',
    descricao: 'Peixes frescos e frutos do mar da pesca artesanal diária entregues com higiene e pontualidade.'
  }
];

export function estaConectadoNaApi() {
  return apiConectada;
}

export async function testarConexaoApi() {
  try {
    const resposta = await fetch(`${API_BASE_URL}/status`, {
      method: 'GET',
      headers: { 'Accept': 'application/json' },
      signal: AbortSignal.timeout(2000)
    });
    apiConectada = resposta.ok;
    return apiConectada;
  } catch (erro) {
    apiConectada = false;
    return false;
  }
}

// Rotinas Locais de Contingência (Fallback)
function obterServicosLocal() {
  const dados = localStorage.getItem(STORAGE_KEY);
  if (!dados) {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(DADOS_INICIAIS));
    return DADOS_INICIAIS;
  }
  try { return JSON.parse(dados); } catch (e) { return []; }
}

function salvarServicoLocal(novoServico) {
  const servicos = obterServicosLocal();
  const servicoCompleto = {
    id: Date.now().toString(),
    dataCadastro: new Date().toISOString(),
    ...novoServico
  };
  servicos.unshift(servicoCompleto);
  localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
  return servicoCompleto;
}

function removerServicoLocal(id) {
```



```
const servicos = obterServicosLocal();
const filtrados = servicos.filter(s => s.id.toString() !== id.toString());
localStorage.setItem(STORAGE_KEY, JSON.stringify(filtrados));
return filtrados;
}

// Operações Assíncronas com Fetch API
export async function obterServicos() {
  try {
    const resposta = await fetch(`${API_BASE_URL}/servicos`, {
      method: 'GET',
      headers: { 'Accept': 'application/json' },
      signal: AbortSignal.timeout(3000)
    });
    if (!resposta.ok) throw new Error(`HTTP ${resposta.status}`);
    const dados = await resposta.json();
    apiConectada = true;
    localStorage.setItem(STORAGE_KEY, JSON.stringify(dados));
    return dados;
  } catch (erro) {
    apiConectada = false;
    console.warn('API Flask offline. Ativando contingência LocalStorage:', erro.message);
    return obterServicosLocal();
  }
}

export async function salvarServico(novoServico) {
  try {
    const resposta = await fetch(`${API_BASE_URL}/servicos`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Accept': 'application/json'
      },
      body: JSON.stringify(novoServico),
      signal: AbortSignal.timeout(4000)
    });
    if (!resposta.ok) throw new Error(`HTTP ${resposta.status}`);
    const criado = await resposta.json();
    apiConectada = true;
    const servicos = obterServicosLocal();
    servicos.unshift(criado);
    localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
    return criado;
  } catch (erro) {
    apiConectada = false;
    console.warn('Falha no envio à API. Salvando localmente:', erro.message);
    return salvarServicoLocal(novoServico);
  }
}

export async function removerServico(id) {
  try {
    const resposta = await fetch(`${API_BASE_URL}/servicos/${id}`, {
      method: 'DELETE',
      headers: { 'Accept': 'application/json' },
      signal: AbortSignal.timeout(4000)
    });
    if (!resposta.ok) throw new Error(`HTTP ${resposta.status}`);
    apiConectada = true;
    removerServicoLocal(id);
    return true;
  }
}
```

```
    } catch (erro) {  
      apiConectada = false;  
      console.warn('Falha na exclusão remota. Removendo localmente:', erro.message);  
      removerServicoLocal(id);  
      return true;  
    }  
  }  
}
```

Fase 4/6: Camada de Visão & Feedbacks de Rede (vitrineView.js)

Responsabilidade Arquitetural: Adicionar componentes visuais de carregamento (exibirSpinner) e indicador de conectividade em tempo real com a API no topo da página.

```
// js/views/vitrineView.js
import { formatarMoeda, formatarTelefone, escapeHtml } from '../utils/formatters.js';

const CORES_CATEGORIA = {
  'Alimentação': 'success',
  'Tecnologia': 'info',
  'Artesanato': 'warning',
  'Serviços Gerais': 'primary',
  'Turismo': 'secondary'
};

export function criarCardHtml(s) {
  const corBadge = CORES_CATEGORIA[s.categoria] || 'primary';
  const telNumeros = (s.telefone || '').replace(/\\D/g, '');

  return `
    <div class="col-md-6 col-lg-4 mb-4">
      <div class="card h-100 shadow-sm border-0 vitrine-card rounded-4 overflow-hidden">
        <div class="card-body p-4 d-flex flex-column">
          <div class="d-flex justify-content-between align-items-center mb-2">
            <span class="badge bg-${corBadge} px-3 py-2 rounded-pill font-outfit">
              <i class="bi bi-tag-fill me-1"></i>${escapeHtml(s.categoria)}
            </span>
            <span class="fw-bold text-success font-outfit fs-5">
              ${formatarMoeda(s.precoBase)}
            </span>
          </div>
          <div>
            <h5 class="card-title fw-bold text-dark mt-2 mb-1">${escapeHtml(s.nome)}</h5>
            <h6 class="text-muted small mb-3">
              <i class="bi bi-geo-alt-fill text-danger me-1"></i>${escapeHtml(s.bairro)}
            </h6>
            <p class="card-text text-secondary small flex-grow-1" style="line-height: 1.5;">
              ${escapeHtml(s.descricao)}
            </p>
            <hr class="my-3 text-muted opacity-25">
            <div class="d-flex justify-content-between align-items-center mt-auto">
              <a href="https://wa.me/55${telNumeros}" target="_blank" class="btn btn-sm btn-outline-success rounded-pill px-3 fw-bold">
                <i class="bi bi-whatsapp me-1"></i>${formatarTelefone(s.telefone)}
              </a>
              <div class="d-flex gap-1">
                <button class="btn btn-sm btn-outline-danger btn-excluir rounded-pill px-2" data-id="${s.id}" title="Remover da vitrine">
                  <i class="bi bi-trash"></i>
                </button>
              </div>
            </div>
          </div>
        </div>
      </div>
    </div>
  `;
}
```

```
        </div>
      </div>
    `;
  }

export function renderizarCards(servicos, containerElement) {
  if (!containerElement) return;
  if (!servicos || servicos.length === 0) {
    containerElement.innerHTML = `
      <div class="col-12 py-5 text-center text-muted">
        <i class="bi bi-inbox fs-1 d-block mb-3 text-secondary opacity-50"></i>
        <h5 class="fw-bold">Nenhum serviço encontrado</h5>
        <p class="small">Cadastre um novo serviço ou altere o filtro de categoria.</p>
      </div>
    `;
    return;
  }
  containerElement.innerHTML = servicos.map(s => criarCardHtml(s)).join('');
}

export function exibirSpinner(containerElement) {
  if (!containerElement) return;
  containerElement.innerHTML = `
    <div class="col-12 py-5 text-center text-primary">
      <div class="spinner-border text-primary" role="status" style="width: 3rem; height: 3rem;">
        <span class="visually-hidden">Carregando dados da API...</span>
      </div>
      <p class="mt-3 text-muted small fw-semibold">Consultando serviços no backend Flask (SQLite)...</p>
    </div>
  `;
}

export function atualizarStatusConexao(isOnline, statusBadgeEl) {
  if (!statusBadgeEl) return;
  if (isOnline) {
    statusBadgeEl.className = 'badge bg-success-subtle text-success border border-success-subtle px-3 py-2 rounded-pill fw-semibold';
    statusBadgeEl.innerHTML = '<i class="bi bi-cloud-check-fill me-1"></i> Backend Online (Flask + SQLite)';
  } else {
    statusBadgeEl.className = 'badge bg-warning-subtle text-warning border border-warning-subtle px-3 py-2 rounded-pill fw-semibold';
    statusBadgeEl.innerHTML = '<i class="bi bi-hdd-network-fill me-1"></i> Modo Offline (LocalStorage)';
  }
}

export function exibirToast(mensagem, tipo = 'success') {
  const toastEl = document.getElementById('toastNotificacao');
  const toastMsgEl = document.getElementById('toastMensagem');
  if (!toastEl || !toastMsgEl) return;
  toastEl.className = `toast align-items-center text-white bg-${tipo} border-0 shadow-lg`;
  toastMsgEl.innerText = mensagem;
  const bsToast = bootstrap.Toast.getOrCreateInstance(toastEl, { delay: 3500 });
  bsToast.show();
}
```

Fase 5/6: Orquestrador Assíncrono da Aplicação (main.js)

Responsabilidade Arquitetural: Coordenar os fluxos de eventos da interface utilizando `await` para aguardar a persistência no banco remoto antes de fechar modais e atualizar a vitrine.

```
// js/main.js
import {
  obterServicos,
  salvarServico,
  removerServico,
  testarConexaoApi,
  estaConectadoNaApi
} from './services/vitrineService.js';

import {
  renderizarCards,
  exibirSpinner,
  atualizarStatusConexao,
  exibirToast
} from './views/vitrineView.js';

document.addEventListener('DOMContentLoaded', async () => {
  const form = document.getElementById('formCadastro');
  const container = document.getElementById('vitrineContainer');
  const filtroCategoria = document.getElementById('filtroCategoria');
  const contadorEl = document.getElementById('totalServicosBadge');
  const statusApiBadge = document.getElementById('statusApiBadge');
  const modalEl = document.getElementById('modalCadastro');
  const btnSalvarTexto = document.getElementById('btnSalvarTexto');
  const btnSalvar = document.getElementById('btnSalvar');

  let cacheServicosAtuais = [];

  async function atualizarInterface() {
    exibirSpinner(container);
    const conectado = await testarConexaoApi();
    atualizarStatusConexao(conectado, statusApiBadge);

    cacheServicosAtuais = await obterServicos();
    const categoriaSelecionada = filtroCategoria ? filtroCategoria.value : 'todas';
    const servicosFiltrados = categoriaSelecionada === 'todas'
      ? cacheServicosAtuais
      : cacheServicosAtuais.filter(s => s.categoria === categoriaSelecionada);

    renderizarCards(servicosFiltrados, container);
    if (contadorEl) {
      contadorEl.innerHTML = `${cacheServicosAtuais.length} Serviços Cadastrados`;
    }
  }

  if (filtroCategoria) {
    filtroCategoria.addEventListener('change', () => {
      const categoria = filtroCategoria.value;
      const filtrados = categoria === 'todas'
        ? cacheServicosAtuais
        : cacheServicosAtuais.filter(s => s.categoria === categoria);
      renderizarCards(filtrados, container);
    });
  }
});
```



```
}

// Inicialização assíncrona
await atualizarInterface();

// Reset do formulário ao abrir modal
if (modalEl) {
  modalEl.addEventListener('show.bs.modal', () => {
    form.reset();
    form.classList.remove('was-validated');
  });
}

// Cadastro de Novo Serviço via HTTP POST
form.addEventListener('submit', async (e) => {
  e.preventDefault();
  if (!form.checkValidity()) {
    e.stopPropagation();
    form.classList.add('was-validated');
    return;
  }

  const getVal = (id) => document.getElementById(id).value.trim();
  const dados = {
    nome: getVal('nome'),
    categoria: document.getElementById('categoria').value,
    bairro: getVal('bairro'),
    precoBase: parseFloat(getVal('precoBase')) || 0,
    telefone: getVal('telefone'),
    descricao: getVal('descricao')
  };

  if (btnSalvar) btnSalvar.disabled = true;
  if (btnSalvarTexto) btnSalvarTexto.innerText = 'Gravando...';

  try {
    await salvarServico(dados);
    const msg = estaConectadoNaApi()
      ? 'Serviço gravado no banco de dados SQLite!'
      : 'Serviço salvo localmente (Modo Offline).';
    exibirToast(msg, 'success');

    const modal = bootstrap.Modal.getOrCreateInstance(modalEl);
    modal.hide();
    await atualizarInterface();
  } catch (erro) {
    exibirToast('Erro ao processar cadastro.', 'danger');
  } finally {
    if (btnSalvar) btnSalvar.disabled = false;
    if (btnSalvarTexto) btnSalvarTexto.innerText = 'Salvar no Banco';
  }
});

// Exclusão Assíncrona via HTTP DELETE
container.addEventListener('click', async (e) => {
  const btnExcluir = e.target.closest('.btn-excluir');
  if (btnExcluir) {
    const id = btnExcluir.getAttribute('data-id');
    if (confirm('Deseja realmente remover este serviço da vitrine?')) {
      await removerServico(id);
      await atualizarInterface();
      exibirToast('Serviço excluído da vitrine.', 'warning');
    }
  }
});
```

```
    }  
  }  
});  
});
```

Fase 6/6: Verificação de Integração & Prova de Contingência

Responsabilidade Arquitetural: Executar o roteiro de testes em dois cenários: com a API online no laboratório e com o servidor desligado comprovando a resiliência offline.

1. Teste Conectado (API Online):

- Inicie o backend com `python app.py` na porta 5000;
- Abra o frontend via Live Server (`http://127.0.0.1:5500/index.html`);
- O badge no topo deve exibir: **Backend Online (Flask + SQLite)**;
- Cadastre um novo serviço e observe a requisição HTTP POST sendo logada no terminal do Flask!

2. Teste de Resiliência (Fallback LocalStorage):

- No terminal do Flask, pressione `Ctrl + C` para desligar o servidor;
- Recarregue a página no navegador com F5;
- O frontend detecta a indisponibilidade e muda o badge para: **Modo Offline (LocalStorage)**;
- Os cards continuam aparecendo normalmente a partir da cópia em cache!

Critério de Conclusão do Roteiro 05 A aplicação demonstrou sucesso na comunicação assíncrona cliente-servidor via Fetch API, realizou persistência no SQLite e provou resiliência arquitetural ao operar em modo de contingência offline.

1.6 Quadro Comparativo: Evolução do Roteiro 04 para o Roteiro 05

Componente	Roteiro 04 (Módulo 2)	Roteiro 05 (Módulo 3)
Ambiente de Dados	LocalStorage (isolado na aba do navegador).	Banco relacional SQLite (<code>vitrine.db</code>) no Flask.
Comunicação	Chamadas de função síncronas em memória.	Chamadas assíncronas de rede com <code>fetch()</code> e <code>await</code> .
Segurança de Rede	Inexistente (tudo no cliente).	Configuração de cabeçalhos CORS (<code>flask-cors</code>).
Experiência (UX)	Resposta imediata sem feedback de rede.	Spinners de espera e badges de status de conectividade.
formatters.js	Funções puras de formatação.	100% idêntico! Prova de desacoplamento limpo.

Parabéns! Você implementou uma arquitetura moderna full-stack desacoplada, apta para o ecossistema profissional e pronta para avançar rumo às Single Page Applications (SPAs) e ao projeto de extensão da SNCT!